

INTRODUCTION TO MACHINE LEARNING

MIHIR RAO

Contents

1	Basics	2
1.1	Training data and hypotheses	2
1.2	Linear classifiers and their parameters	3
1.3	Why the dot product describes the boundary	4
1.4	Loss: measuring a prediction's cost	5
1.5	Training error and test error	5
1.6	Learning a classifier	6

LECTURE 1: BASICS

MACHINE LEARNING uses data to make decisions. Applications include exoplanet detection, health prediction, legal analysis, labor statistics, phishing detection, facial recognition, and satellite-assisted lending. Studying it helps us apply these methods, understand them, and evaluate their limits.

Given examples with known labels, how can we label a new example? We need data, possible prediction rules, a measure of mistakes, and an algorithm to choose a rule.

1.1 Training data and hypotheses

A data point has measured attributes and a label we want to predict. We put the attributes into a vector so that a rule can use them together.

Definition 1.1 [LABELED TRAINING DATA]. For each $i \in \{1, \dots, n\}$, the FEATURE VECTOR is

$$x^{(i)} = (x_1^{(i)}, \dots, x_d^{(i)})^T \in \mathbb{R}^d,$$

and its LABEL is $y^{(i)} \in \{-1, +1\}$. The TRAINING DATA consist of the n labeled examples

$$\mathcal{D}_n = \{(x^{(1)}, y^{(1)}), \dots, (x^{(n)}, y^{(n)})\}.$$

The superscript (i) identifies the example; the subscript identifies a feature. Thus $x_2^{(3)}$ is the second feature of the third example, rather than a power of x_2 . The transpose makes each feature vector a column vector.

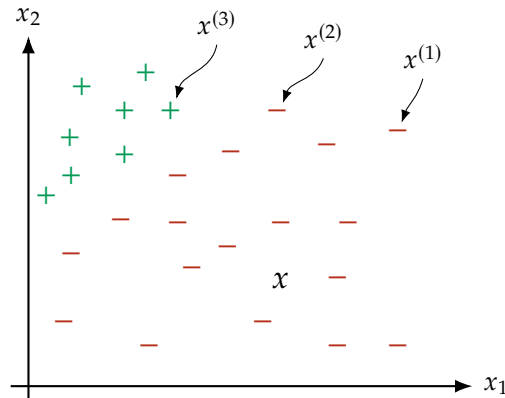


Figure 1: Schematic labeled data in two dimensions. Each position is a feature vector; its + or - gives the observed label. The point marked x still needs a prediction. The coordinates illustrate structure, not measurements.

Definition 1.2 [HYPOTHESIS]. A HYPOTHESIS, or prediction rule, is a function

$$h : \mathbb{R}^d \longrightarrow \{-1, +1\}.$$

Given a feature vector x , it returns a predicted label $h(x)$. In this binary labeling problem, such a rule is also called a CLASSIFIER.

Example: *Always predict +1.* The rule $h(x) = +1$ for every x is a valid hypothesis: it assigns an allowed label to every input. Being a hypothesis does not make it a good one. This rule gets every example with label -1 wrong.

The task is to predict labels for new feature vectors, rather than merely to describe labels already present in $\mathcal{D}_n$.

1.2 Linear classifiers and their parameters

We need to decide which prediction rules the learning algorithm may consider. A HYPOTHESIS CLASS $\mathcal{H}$ is a set of hypotheses. A useful first class assigns $+1$ to one side of a line and -1 to the other. In d dimensions, the corresponding boundary is a hyperplane.

Definition 1.3 [LINEAR CLASSIFIER]. Let $\theta \in \mathbb{R}^d$ and $\theta_0 \in \mathbb{R}$. A LINEAR CLASSIFIER is

$$h(x; \theta, \theta_0) = \text{sign}(\theta^\top x + \theta_0) = \begin{cases} +1, & \theta^\top x + \theta_0 > 0, \\ -1, & \theta^\top x + \theta_0 \leq 0. \end{cases}$$

Here the sign convention assigns -1 when the score is exactly zero. The PARAMETERS are the vector θ and the scalar θ_0 . The class $\mathcal{H}$ contains all rules of this form.

The score $\theta^\top x + \theta_0$ decides the label. When $\theta \neq 0$, the DECISION BOUNDARY is

$$\{x : \theta^\top x + \theta_0 = 0\}.$$

The vector θ sets the boundary's orientation and points toward the positive side. The scalar θ_0 sets its offset. For $\theta = 0$, the score is constant, so the rule predicts one label everywhere; there is no ordinary separating hyperplane. The geometric formulas below assume $\theta \neq 0$.

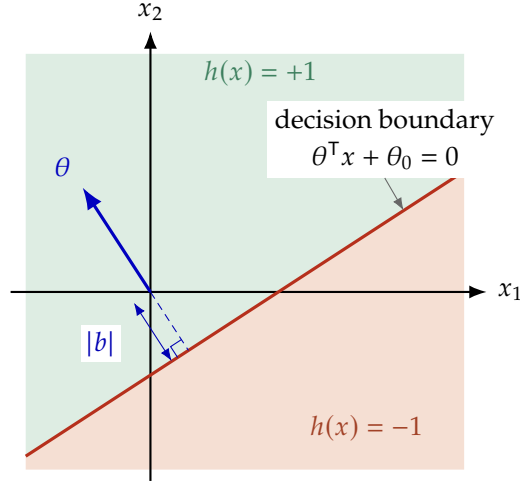


Figure 2: A schematic linear classifier. The normal θ points toward the positive region. The boundary is included in the negative region by the stated sign convention. Its offset is measured along the unit normal.

1.3 Why the dot product describes the boundary

The quantity $\theta^T x$ measures how far x extends in the direction of θ , up to a scale factor. Write

$$u = \frac{\theta}{\|\theta\|}, \quad \|\theta\| = \sqrt{\theta^T \theta}.$$

Since u has length one, the SIGNED PROJECTION of x along this direction is

$$u^T x = \frac{\theta^T x}{\|\theta\|}.$$

Positive values are in the direction of θ ; negative values are in the opposite direction. Dividing the boundary equation by $\|\theta\|$ gives

$$\frac{\theta^T x}{\|\theta\|} = -\frac{\theta_0}{\|\theta\|}.$$

Thus every point on the boundary has the same signed projection. If b denotes that projection, then

$$b = -\frac{\theta_0}{\|\theta\|}, \quad \theta^T x + \theta_0 > 0 \iff u^T x > b.$$

The positive region lies beyond this threshold in the direction of θ . The perpendicular distance from the origin to the boundary is $|b|$; b itself is signed.

For two boundary points p and q , subtract their equations:

$$(\theta^T p + \theta_0) - (\theta^T q + \theta_0) = 0 \implies \theta^T (p - q) = 0.$$

Since $p - q$ runs along the boundary, its zero dot product with θ establishes perpendicularity.

1.4 Loss: measuring a prediction's cost

A classifier should work well on future data. Before comparing classifiers, we must specify what a mistake costs.

Definition 1.4 [Loss]. A LOSS $L(g, a)$ gives the cost of a guessed label g when the actual label is a . For example, the ZERO-ONE LOSS is

$$L(g, a) = \begin{cases} 0, & g = a, \\ 1, & g \neq a. \end{cases}$$

Zero-one loss treats the two kinds of wrong prediction equally. A different task may require different penalties.

Example: *Asymmetric mistakes.* Consider

$$L(g, a) = \begin{cases} 1, & g = +1, a = -1, \\ 100, & g = -1, a = +1, \\ 0, & g = a. \end{cases}$$

Predicting -1 when the actual label is $+1$ costs one hundred times as much as the reverse mistake. A correct prediction costs zero in either class.

The order of the arguments matters: the first is the guess and the second is the actual label. With asymmetric loss, exchanging them changes the cost.

1.5 Training error and test error

To evaluate a classifier on several examples, average its losses. The TRAINING ERROR on the data used for learning is

$$\mathcal{E}_n(h) = \frac{1}{n} \sum_{i=1}^n L(h(x^{(i)}), y^{(i)}).$$

With zero-one loss, the sum counts misclassified examples, so $\mathcal{E}_n(h)$ is the fraction of training examples predicted incorrectly. With asymmetric loss, it is an average cost, rather than a fraction of mistakes.

The TEST ERROR on n' new examples, indexed $n + 1, \dots, n + n'$, is

$$\mathcal{E}(h) = \frac{1}{n'} \sum_{i=n+1}^{n+n'} L(h(x^{(i)}), y^{(i)}).$$

The two formulas use the same loss but different data. The test average measures performance on the new examples, which is the behavior we want to assess.

When choosing from training data, a natural preference is h over $\tilde{h}$ if $\mathcal{E}_n(h) < \mathcal{E}_n(\tilde{h})$. This compares their training performance. It does not establish the same ordering of their test errors.

1.6 Learning a classifier

A classifier takes an input vector and returns a label. A **LEARNING ALGORITHM** has a different input and output: it takes training data and returns a classifier from the chosen class. For linear classifiers, this means choosing θ and θ_0 using $\mathcal{D}_n$.

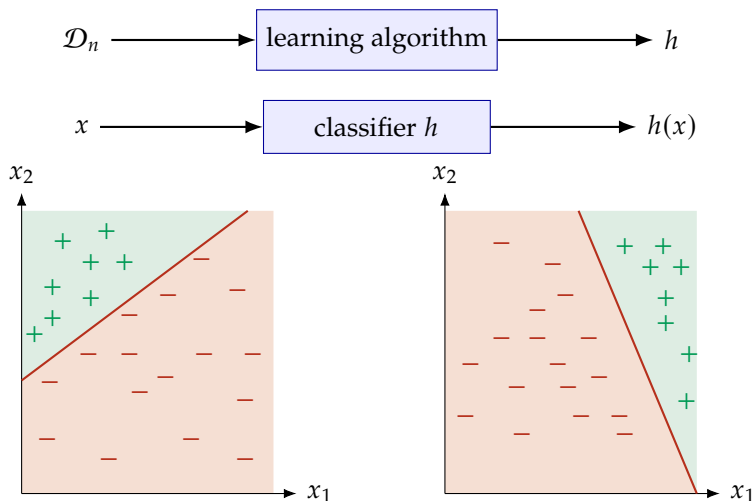


Figure 3: Learning chooses a rule from labeled data; prediction applies that rule to a feature vector. The two schematic data arrangements below admit different separating lines, illustrating why the chosen parameters depend on the training data.

An example: search a random candidate pool

One conceptual learning algorithm constructs many candidate classifiers and selects the one with the smallest training error. Let $M = 10^{12}$, and randomly sample a fixed pool of M parameter pairs. For an integer $1 \leq k < M$, compare only the first k candidates. Write

$$h^{(j)}(x) = h(x; \theta^{(j)}, \theta_0^{(j)}), \quad j = 1, \dots, M.$$

The superscript (j) now indexes candidates, while (i) indexes data points.

Algorithm 1.1: Select a classifier from a random candidate pool

Input: Training data $\mathcal{D}_n$; integer $1 \leq k < M$, where $M = 10^{12}$.

Output: A classifier with smallest training error among the first k candidates.

```
1 for  $j \leftarrow 1$  to  $M$  do
2   | sample a parameter pair  $(\theta^{(j)}, \theta_0^{(j)})$ 
3   | Set  $h^{(j)}(x) \leftarrow h(x; \theta^{(j)}, \theta_0^{(j)})$ 
4   |  $j^* \leftarrow \arg \min_{j \in \{1, \dots, k\}} \mathcal{E}_n(h^{(j)})$ 
5 return  $h^{(j^*)}$ 
```

If several candidates have the same minimum error, choose the smallest index. The data determine which candidate wins within the pool. The number k is a **HYPERPARAMETER**: it controls the learning procedure, rather than appearing in the classifier's prediction formula. The classifier's parameters remain θ and θ_0 .

Proposition 1.5 [COMPARING ONE CANDIDATE WITH TWO]. *For the same data, loss, and fixed candidate pool, the rule selected with $k = 2$ has training error no larger than the rule selected with $k = 1$.*

Proof. The first candidate remains available when the second is added. With $k = 1$, the selected error is $\mathcal{E}_n(h^{(1)})$. With $k = 2$, it is

$$\min\{\mathcal{E}_n(h^{(1)}), \mathcal{E}_n(h^{(2)})\} \leq \mathcal{E}_n(h^{(1)}).$$

It decreases if the second candidate has strictly smaller error; otherwise it stays the same. $\square$

This comparison requires a shared pool, so the first candidate is the same in both runs. It does not compare independent random draws and does not guarantee an improvement in test error.

Revision. Data pair features with labels. A hypothesis predicts a label; a hypothesis class lists allowed rules. Linear classifiers use the sign of $\theta^\top x + \theta_0$. Training and test error average the chosen loss on different data. Learning uses training data to choose classifier parameters.

Common mistakes: confusing example indices with feature indices, assigning +1 on a zero score, swapping guess and actual label in an asymmetric loss, or treating smaller training error as a guarantee of smaller test error.

Check yourself: Why is θ normal to the boundary? How does asymmetric loss distinguish mistakes? Why can adding a candidate never increase minimum training error in a fixed pool? How does k differ from θ and θ_0 ?

Source. Tamara Broderick, MIT 6.036/6.862, *Introduction to Machine Learning*, Fall 2020, [Lecture 1: Basics](#), PDF pp. 34–185 (numbered slides 5–10), via the [official lecture collection](#). Repeated animation builds are consolidated and administration omitted. The geometric steps and the candidate-comparison argument are completed here; the figures are original schematic redraws.